Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Сборка образа: Dockerfile, оптимизация, healthcheck и теги

Обучающийся: Мануковская Дарья Михайловна

Номер ИСУ: 466625

Факультет: Факультет прикладной информатики

Группа: К3221

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Мануковская Д. М.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Ход выполнения **4**

1.1 Первый рабочий образ 4

1.2 Оптимизация образа 5

1.3 .dockerignore - чистота в образе 6

1.4 Healthcheck - готовность к оркестрации 6

1.5 Теги и финальный артефакт 8

1.6 Ошибки и анализ результатов 8

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Собрать собственный образ приложения assist-py, уменьшить его размер и добавить проверку здоровья. Для этого нужно описать сборку в Dockerfile, убрать лишние файлы и кэш пакетов, а затем научить Docker проверять, что сервис отвечает. Образ должен запускаться с префиксом manuk на порту 5455.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	466625
Персональный порт	5455
Персональный префикс	manuk
Секретный ключ	secret25manuk

Результат хранится в ветке `lab-2` своего форка: https://github.com/wecanallreborn/KIOP/tree/lab-2.

1 Ход выполнения

1.1 Первый рабочий образ

Клонировали репозиторий с готовым приложением.

```
git clone https://github.com/FilBTi/assist-py solutions/assist-py
rm -rf solutions/assist-py/.git
cd solutions/assist-py
```

Создали docker файл в корне проекта assist-py. Написали конфигурацию базового образа.

```
FROM python:3.11-slim
LABEL maintainer="manuk@lab.local"
LABEL version="1.0"
WORKDIR /app
ENV PORT=5455
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE ${PORT}
CMD uvicorn app.main:app --host 0.0.0.0 --port ${PORT}
```

Собираем образ при помощи команды.

```
docker build -t manuk-assistant:v1 .
```

Запускаем контейнер с переменным окружением и персональными параметрами.

```
docker run -d \
  --name manuk-assistant-1 \
  -p 5455:5455 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  manuk-assistant:v1
```

Проверяем при помощи команд работоспособность контейнера, созданного по ранее написанному образу. Терминальный вывод представлен на рисунке 1.

```
curl http://localhost:5455/
curl http://localhost:5455/me
curl -X POST http://localhost:5455/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР2: сборка образа"}'
curl http://localhost:5455/notes
curl http://localhost:5455/secret/secret25manuk
```



Рисунок 1 — Результат работы команд

1.2 Оптимизация образа

Для уменьшения размера были уже добавлены следующие строчки в изначальный файл. В новой версии поставим без флага кэша.

```
RUN pip install --no-cache-dir -r requirements.txt
```

Проверим насколько флаг влияет на вес образа.

```
docker build -t manuk-assistant:v2 -f Dockerfile.optimized .
docker images | grep manuk-assistant
```

На 12.3MB образ с флагом оказался больше. (рисунок 2)

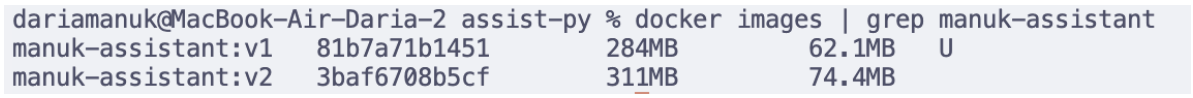


Рисунок 2 — Результат работы команд

При изменении слоя докер пересобирает все слои с него и ниже по коду (выше по слоям). То есть при изменении любых файлов будут заного

переустанавливаться библиотеки из requirements, тк эта команда выполняется после измененного слоя. Копируя отдельно файл, предотвращается эта проблема. Для ранее выполненны теже команды, то в логах CACHED и они не выполняются повторно.

1.3 .dockerignore - чистота в образе

Создали в корне проекта файл .dockerignore.

```
__pycache__/
*.pyc
*.pyo
.git
.gitignore
.venv/
venv/
.env
*.log
Dockerfile*
README.md
```

Далее собрали новый образ без этих файлов.

```
docker build -t manuk-assistant:v3 .
```

Файлы гита создают потенциальную опасность из-за хранения всей истории коммитов, при неосторожности: если были закоммичены файлы с чувствительными данными, они останутся там. Без .dockerignore выгружаются файлы виртуального окружения, логи, сам Dickerfile (который уже не нужен), файлы гита, а так же секреты .env, что недопустимо.

1.4 Healthcheck - готовность к оркестрации

Добавили в Dockerfile.optimized (добавив и удаленный флаг). Ставим curl для самопроверки контейнера изнутри (в слим версии этой команды нет), все выполняется одним run для удаления списка пакетов в одном слое, для оптимизации веса также ставятся лишь обязательные пакеты.

```
RUN apt-get update \
```

```
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s
 ↪  --retries=3 \
CMD curl -f http://localhost:${PORT}/health || exit 1
```

Собираем финальный образ и сравниваем вес (рисунок 3).

```
docker build -t manuk-assistant:final -f Dockerfile.optimized .
docker images | grep manuk-assistant
```

```
dariamanuk@MacBook-Air-Daria-2 assist-py % docker images | grep manuk-assistant
manuk-assistant:final   90ddd45cb68a        304MB         67.1MB
manuk-assistant:v1      81b7a71b1451        284MB         62.1MB   U
manuk-assistant:v2      ca712ca39adf        311MB         74.4MB
manuk-assistant:v3      85bf73c2899c        284MB         62.1MB
```

Рисунок 3 — Результат работы команд

Проверяем статус, терминальный вывод представлен на рисунке 4.

```
docker run -d \
  --name manuk-assistant-final \
  -p 5455:5455 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  manuk-assistant:final
sleep 10
docker ps --filter "name=manuk-assistant-final" --format "table
 ↪  {{.Names}}\t{{.Status}}"
```

```
dariamanuk@MacBook-Air-Daria-2 assist-py % docker run -d \
  --name manuk-assistant-final \
  -p 5455:5455 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  manuk-assistant:final
sleep 10
docker ps --filter "name=manuk-assistant-final" --format "table {{.Names}}\t{{.Status}}"

546093e16c2462d284402ca0acf912aa8316047114f543c42ff6d0961155920a
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint manuk-assistant-final (fb540421dd4c49ec
ad3c1dc6478529ab2802c49b1ee95b99df395816420edb7a): Bind for 0.0.0.0:5455 failed: port is already allocated

Run 'docker run --help' for more information

What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
NAMES     STATUS
```

Рисунок 4 — Результат работы команд

1.5 Теги и финальный артефакт

```
docker build -t manuk-assistant:final -f Dockerfile.optimized .
docker tag manuk-assistant:final manuk-assistant:1.0.0
docker tag manuk-assistant:final manuk-assistant:$(date +%Y%m%d)
```

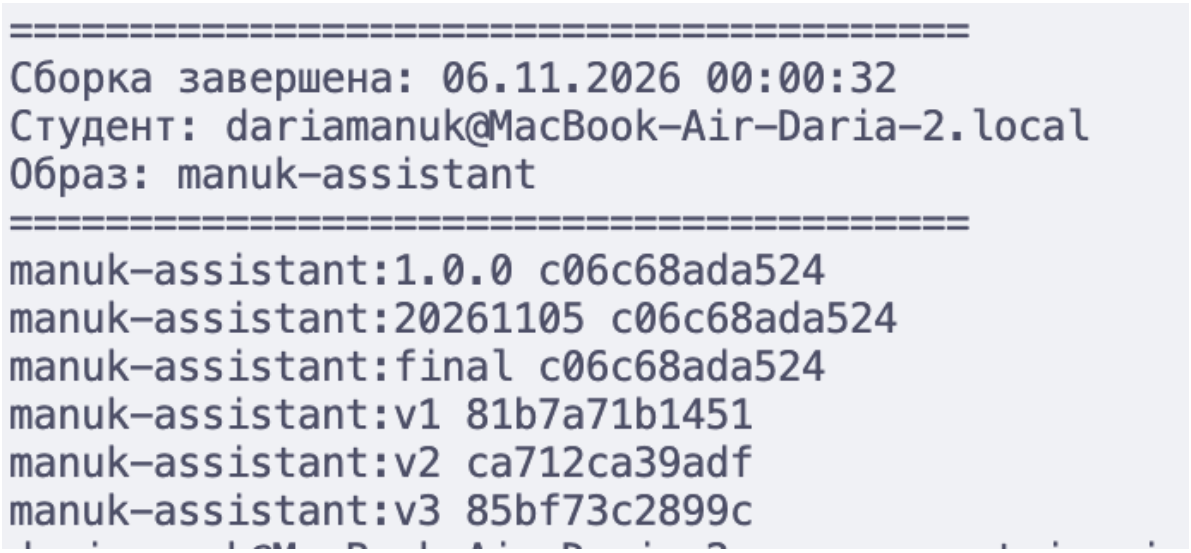


Рисунок 5 — Защита от копирования

1.6 Ошибки и анализ результатов

При сборке первого образа получили предупреждение из-за использования shell формы вместо exec (рисунок 6). Такой синтаксис необходим для правильной подстановки значения порта как переменной.

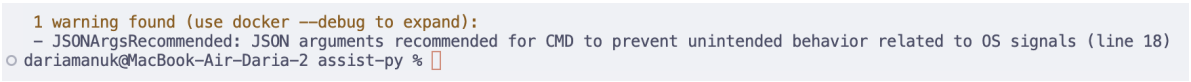


Рисунок 6 — Предупреждение

Запуск финального контейнера с первого раза не состоялся, тк порт держал контейнер из 1 этапа (рисунок 7). Удалили оба контейнера и перезапустили команды.

```
dariamanuk@MacBook-Air-Daria-2 assist-py % docker run -d \
  --name manuk-assistant-final \
  -p 5455:5455 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  manuk-assistant:final
sleep 10
docker ps --filter "name=manuk-assistant-final" --format "table {{.Names}}\t{{.Status}}"

6dee06fcd1c06e1dde3f79d0e1224d25e2460f743b0d91e206de4c0dee381719
NAMES                  STATUS
manuk-assistant-final   Up 10 seconds (healthy)
```

Рисунок 7 — Неудачный запуск контейнера

ЗАКЛЮЧЕНИЕ

Собран собственный образ приложения assist-py, уменьшен его размер и добавлена проверка здоровья. Сборка описана в Dockerfile, лишние файлы и кэш пакетов исключены, Docker проверяет, что сервис отвечает. Образ запускается с префиксом manuk на порту 5455 и получает статус healthy.